

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

ЗВІТ

з виробничої практики з відривом від навчання

з освітньо-професійної програми «Програмна інженерія»

за спеціальністю 121 Інженерія програмного забезпечення

Виконала
Студентка 3-го курсу
ЧЕЛУШКІНА Валерія
Олександрівна

Керівник практики від кафедри:
асистент
СУПРУН Олег Олексійович

Керівник практики від бази
практики:
ЛЕВКІВСЬКИЙ Євген Сергійович

Засвідчую, що в цій роботі немає
запозичень з праць інших авторів
без відповідних посилань

Студентка _____

Київ – 2026

Зміст

<i>ВСТУП</i>	<i>3</i>
<i>ПОСТАНОВКА ЗАВДАННЯ</i>	<i>4</i>
<i>Мета та постановка задачі</i>	<i>4</i>
<i>Розподіл обов'язків у команді</i>	<i>4</i>
<i>Використаний стек технологій</i>	<i>5</i>
<i>Загальна архітектура системи</i>	<i>6</i>
<i>РЕАЛІЗАЦІЯ</i>	<i>7</i>
<i>Модуль скрапінгу вакансій</i>	<i>7</i>
<i>Фасад та паралельний збір даних</i>	<i>9</i>
<i>Модуль збереження даних</i>	<i>10</i>
<i>HTTP-API та фільтрація джерел</i>	<i>10</i>
<i>Аналітика даних</i>	<i>10</i>
<i>ТЕСТУВАННЯ, РЕЗУЛЬТАТИ</i>	<i>12</i>
<i>Тестування скраперів та обробка помилок</i>	<i>12</i>
<i>Перспективи розвитку проєкту</i>	<i>12</i>
<i>Результати</i>	<i>12</i>
<i>ВИСНОВКИ</i>	<i>15</i>

ВСТУП

У межах виробничої практики було надано завдання розробити веб-застосунок для аналізу та перегляду ринку ІТ-вакансій. Проєкт більше розрахований на початківців в цій сфері, які хочуть орієнтуватися в актуальних технологіях, вакансіях та рівнях зарплат, хоча може бути корисним і більш досвідченим працівникам. Система збирає вакансії з найбільш популярних платформ, агрегує дані та надає певні візуалізації даних: які скіли найбільше потребують на певну посаду, які рівні досвіду потрібні, які зарплатні діапазони пропонуються.

Було прийнято рішення зробити цей проєкт у команді, із одnogрупницею, яка знаходиться на цій же базі практики. Корисно вчитися працювати в команді якомога раніше, оскільки в реальних професіях ІТ-сфери це велика частина роботи і це те, з чим треба буде стикатися кожного дня. Протягом всієї роботи була постійна комунікація із іншою учасницею команди, щоб бути в курсі, на якому етапі кожна з нас знаходиться і обговорити подальші дії, обговорити можливий функціонал, який можна ще додати і зробити застосунок більш зручним для користувача.

За час практики було пройдено повний цикл розробки навчального продукту: від аналізу проєкту та вивчення нового функціоналу потрібного для роботи, формулювання вимог та вибору стеку до реалізації функціоналу, налагодження інтеграції фроненда й бекенда, написання документації та тестування готового продукту.

Моєю роботою в команді насамперед було відповідати за бекенд-частину проєкту: модулі скрапінгу, фасад для збору й очищення даних, API для фроненда, базову аналітику, тестування, а також частково за структуру проєкту та організацію коду.

Головною метою роботи було не лише реалізувати необхідний функціонал, а й застосувати навички, що раніше були мною вивчені, зіштовхнутися з тим, що раніше не вивчала і навчитися правильно це робити, правильно побудувати роботу в команді.

ПОСТАНОВКА ЗАВДАННЯ

Мета та постановка задачі.

Метою проєкту є створити зручний інструмент для аналізу IT-ринку праці. Користувач вводить ключове слово, наприклад «Front-end developer» або «Python», обирає з яких саме джерел він хоче збирати вакансій (запропоновано три джерела – DOU, Djinni, LinkedIn) та кількість вакансій, які хоче бачити користувач і потрібно проаналізувати. Система збирає дані з обраних сайтів, нормалізує, а потім відображає:

- список вакансій у вигляді структурованих карток, в яких вказана найголовніша інформація – посада, компанія, локація, скіли, посилання, та за наявності зарплатня та потрібний досвід;
- діаграми: середня зарплата (де вона була вказана), середній досвід, найчастіше згадувані скіли, топ локацій;
- панель збережених вакансій, де користувач може локально зберігати потрібні вакансії.

Найголовніші вимоги, що були сформовані для реалізації цього проєкту:

- мати можливість обирати як мінімум із трьох різних популярних джерел;
- можливість легко додавати нові джерела в майбутньому;
- миттєве оновлення сторінки пошуку без повного перезавантаження для зручності користувача;
- простий інтерфейс, темна/світла тема, локальне збереження обраних вакансій.

Розподіл обов'язків у команді

На початку практики ми з однією групою сформували список задач та поділили відповідальність. На себе я взяла:

- проєктування та реалізацію модуля скрапінгу для DOU, Djinni, LinkedIn;
- розробку фасаду як єдиної точки входу до бекенд-логіки;

- модуль збереження результатів у JSON та базову очистку та нормалізацію даних;
- проєктування та реалізацію HTTP-API на основі Flask (/api/search, /search);
- частину аналітики (розробка моделей даних та алгоритмів обчислення показників, які згодом візуалізує фронтенд).

Друга учасниця зосередилась на:

- створенні HTML/CSS сторінки, стилізації інтерфейсу та адаптивного дизайну;
- розробці JavaScript-логіки для інтерактивного пошуку, побудови карток вакансій та панелі збережених вакансій (localStorage);
- підключенні бібліотеки Chart.js та побудові діаграм для аналізу;
- покращенні UX (валідація форм, перемикач теми, обробка помилок на стороні клієнта).

Частину рішень ми приймали спільно – формат JSON-об'єкта вакансії, вигляд API, загальний вигляд веб-застосунку, структуру файлів та повідомлень про помилки.

Використаний стек технологій

Для реалізації своєї частини роботи я застосувала такі технології та бібліотеки:

- Python 3.13 - основна мова бекенду;
- Flask – мікрофреймворк для побудови веб-застосунку та REST-ендпоінтів;
- Requests – для HTTP-запитів до сайтів DOU та Djinni;
- BeautifulSoup – для HTML-парсингу сторінок вакансій та оголошень;
- Playwright – для скрапінгу LinkedIn, який активно використовує JavaScript та динамічне завантаження контенту;
- concurrent.futures.ThreadPoolExecutor – для паралельного збору вакансій з різних джерел;

- JSON – для збереження результатів у файли з таймштампами;
- localStorage (на стороні фронтенду, але з урахуванням формату бекенду)
– для збереження обраних вакансій.

Загальна архітектура системи

Набір стратегій DouScraper, DjinniScraper, LinkedInScraper, що реалізують спільний інтерфейс BaseScraper. Кожна стратегія вміє зібрати список вакансій для заданого ключового слова.

Фасад – координує виклик скраперів, паралельно збирає дані, виконує очистку та унікалізацію записів, передає результати до сховища.

Рівень збереження – клас DataStorage, який зберігає результати останнього запиту в JSON-файл із відміткою часу й ключовим словом.

Flask – маршрути / (рендер початкової сторінки), /search (синхронний POST-пошук) та /api/search (AJAX-пошук, який повертає JSON).

РЕАЛІЗАЦІЯ

Під час ознайомлення з проектом та тим, як правильно його буде реалізувати, я зіштовхнулась із новими для себе елементами та інструментами – зокрема скрапінг, тобто отримання інформації із сайтів, та робота із Flask для реалізації HTTP-API. Тому певний час перед початком реалізації проекту я вивчала та опановувала ці інструменти, вчилась, як правильно їх використовувати.

Модуль скрапінгу вакансій

Першим етапом роботи було розробити скрапінг для трьох вебсайтів: DOU, Djinni and LinkedIn. Хоч з цих вебсайтів я отримую однакову інформацію, структура сторінок чи наявність API відрізняється між цими трьома. Тому потрібно було знайти підхід до кожного індивідуально. Я аналізувала структуру та поведінку кожної сторінки, щоб для кожного сайту реалізувати правильний скрапінг.

Використовувала патерн Strategy – поведінковий патерн проектування, який визначає сімейство схожих алгоритмів і розміщує кожен з них у власному класі, щоб у трьох різних скраперів сайтів була однакова структура.

Спочатку розробила абстрактний клас BaseScraper, який визначає:

вхід: ключове слово (keyword) та бажаний ліміт вакансій;

вихід: список словників однакової структури із полями title, company, salary, location, experience, skills, source, link.

DouScraper:

За ключовим слово формується URL. HTML-сторінка завантажується через requests з вказаним User-Agent. За допомогою бібліотеки BeautifulSoup знаходить елементи вакансій та зчитує всі потрібні поля. Для кожної сторінки додатково відкривається повна сторінка цієї вакансії, витягується повний опис та знаходить скіли, що потребують.

DjinniScraper:

Djinni повертає структуровані дані у вигляді JSON-LD у <script type="application/ld+json">. Я використала бібліотеку json для парсингу цього

блоку. Для кожної вакансії дістаються потрібні поля з JSON-масиву. Окремим запитом завантажується сторінка вакансії, з якої додатково витягуються локація та досвід, у Djinni вони чітко прописані в кожній вакансії.

LinkedInScraper:

Оскільки цей сайт доволі захищений, активно використовує JavaScript та динамічне завантаження, використовувати тут BeautifulSoup вже не вийде, тому я застосувала Playwright. Через sync_playwright запускається браузер Chromium у headless-режимі. По ключовому слову відкривається сторінка пошуку вакансій, зчитуються картки та потрібні поля з них. Для кожної вакансії відкривається окрема вкладка, де завантажується повний опис.

Під час роботи із **LinkedIn** зіштовхнулась із проблемою UI Interception – перехоплення подій інтерфейсу модальними вікнами. Під час переходу на сторінку певної вакансії я не могла зібрати потрібну мені інформацію. Для вирішення було використано механізм DOM Event Dispatching, що дозволило програмі взаємодіяти з елементами сторінки в обхід перекриваючих елементів, в даному випадку модальних вікон авторизації.

В кінці процесу в кожному скрапері зберігаються знайдені дані та складається словник вакансії, додаючи позначку джерела та посилання.

При відкритті сторінки не завантажуються відразу всі можливі вакансії по ключовому слову, а спочатку лише обмежена кількість. Наприклад на сайті DOU завантажується 20 вакансій і щоб побачити більше потрібно натиснути на кнопку; на Djinni і LinkedIn 15 і 25 на одній сторінці відповідно, потім потрібно переходити на іншу сторінку для продовження перегляду. Оскільки користувачу забезпечується якомога більша функціональність та доступність, я розібралась із цією проблемою і оптимізувала код так, щоб користувач бачив стільки вакансій скільки йому потрібно. Глобальні зміни були тільки в коді для сайту DOU – я в ході роботи змінила роботу бібліотеки BeautifulSoup на аналогічну роботу до LinkedIn, тобто на Playwright (щоб можна було віртуально натиснути на кнопку та завантажити більше вакансій).

Також під час перегляду структури сайтів я помітила, що на всіх трьох можна фільтрувати за потрібним досвідом. Тому згодом я і це інтегрувала у програму, користувач під час обирання форми може обрати який досвід з переліку і йому будуть показувати тільки відповідні позиції.

Фасад та паралельний збір даних

Застосувала патерн Facade – структурний патерн проектування, який надає простий інтерфейс до складної системи класів, бібліотеки або фреймворку.

Я створила відповідний клас, щоб не дублювати код в різних частинах і щоб зручніше відбувався виклик скраперів. Він інкапсулює список стратегій та надає метод: `fetch_and_store_vacancies(keyword, limit, sources=None)`

Цей метод:

- Формує список активних скраперів. Якщо користувач у застосунку обрав лише частину джерел (наприклад, тільки DOU і Djinni), фасад відфільтровує стратегії за їхнім `source_id`. Якщо список пустий, використовуються всі джерела.
- Викликає допоміжний метод `_collect_parallel`, який за допомогою `ThreadPoolExecutor` запускає збір із кожного джерела в окремому потоці.
- Об'єднує результати в один список, передає їх у `_clean_data` для очистки, а потім у `DataStorage` для збереження.
- Повертає очищений список та шлях до JSON-файлу з результатами.

Був реалізований паралельний збір даних (multithreading) для оптимізації. Таким чином час очікування користувача збору даних значно скорочується (додатково створила таймер та порівняла і дійсно, з багатопотоковістю набагато швидше збираються дані, що є логічно)

В `_clean_data` я реалізувала усунення дублікатів вакансій на основі поля `link`, нормалізацію відсутніх скілів та обрізання зайвих пробілів у назвах.

Модуль збереження даних

Ще одним моїм завданням було збереження результатів пошуку в окремі файли, щоб можна було згодом повторно проаналізувати вже зібрані дані та не втрачати результати при перезапуску сервера. Для цього я реалізувала клас `DataStorage`, метод `save_to_json` якого:

- створює директорію `data_results`, якщо її ще немає;
- формує ім'я файлу за шаблоном `<keyword>_<YYYYMMDD_NHMM>.json`;
- записує список словників вакансій у JSON з кодуванням UTF-8 та відступами для читабельності;
- повертає шлях до файлу або `None` у разі помилки, при цьому виводячи діагностичне повідомлення в консоль.

Можна це ще оптимізувати та ще додати збереження даних у реляційну базу типу SQLite або PostgreSQL для складніших запитів.

HTTP-API та фільтрація джерел

Я спроектувала два основні routes у Flask:

`/search` (POST) – використовується при звичайному сабміті форми; він завантажує `index.html` з уже заповненим списком `vacancies`;

`/api/search` (GET) – використовується JavaScript-кодом через `fetch` для динамічного пошуку, повертає JSON.

Обидва читають параметри `keyword` та `limit`; читають список обраних джерел `sites` (перелік `dou`, `djinni`, `linkedin`); передають параметр `sources` у фасад, який уже вирішує, які стратегії задіяти.

Аналітика даних

Хоча безпосереднє відображення діаграм реалізоване у фронтенді, я також брала участь у розробці логіки аналітики. Була передбачена можливість підрахунку середньої зарплати та досвіду, розподілу скілів, кількості вакансій за локаціями.

Можна звісно більш оптимізувати та придумати більше категорій за якими можна аналізувати дані. Також за потреби ці дані зможуть передаватись на сервер, наприклад якщо забагато вакансій або результати ці треба зберегти.

ТЕСТУВАННЯ, ПЕРСПЕКТИВИ РОЗВИТКУ ТА РЕЗУЛЬТАТИ

Тестування скраперів та обробка помилок

Під час розробки модулів скрапінгу я стикалася з типовими проблемами, такими як зміна у верстці сторінки, HTTP-помилки (тимчасова недоступність, блокування за User-Agent), неповні або некоректні дані (багато де відсутні зарплатня та досвід)

Для їх вирішення я додала перевірку статус-кодів та повернення порожніх списків замість падіння програми.

Для відсутніх полів використовую значення за замовчуванням (“Not specified”, “Look in the description”), щоб структура даних була сталою.

На рівні Flask-додатку я також перевіряла: коректність обробки порожніх ключових слів; роботу з різними значеннями ліміту, чи правильно все працює; реакцію API на відсутність джерел – у такому разі використовується повний набір.

Перспективи розвитку проєкту

Які перспективи для цього проєкту я бачу наразі:

1. Підключення додаткових джерел для збору інформації. Через те, що використовуються патерни Стратегія та Фасад, це реалізувати на складно і програму радикально переробляти не треба, лише додати потрібні джерела і адаптувати збір даних під них.
2. Використання бази даних для збереження історичних даних та спостереження, як поведуть себе різні категорії.
3. Можливість створювати акаунт окремий, де можна переглядати збережені вакансії або історію вакансій. Наразі можна переглядати збережені вакансії лише локально.

Результати

Декілька скрінів, як виглядає веб-застосунок на фінальному етапі проєкту:

IT Job Market Analyzer

Saved vacancies: 1 [View](#)

Number of vacancies shown: 1 [🔊](#) [🔍](#) Experience [▼](#)

[DOU](#) [DJINNI](#) [LINKEDIN](#)

Found 3 vacancies.

01 [DOU]	PHP Senior Developer Not specified. Peratera за кордоном, віддалено Look in the description.	REQUIRD SKILLS: SQL AWS	Visit Save
02 [DJinni]	Junior PHP Developer (Laravel / Symfony) Not specified partnerdevs Україна Виключно від 0.5 років досвіду	REQUIRD SKILLS: React SQL Docker AWS PHP MongoDB	Visit Saved
03 [LinkedIn]	PHP Web Developer By agreement. COMSOL, Inc. Burlington, MA Look in the description	REQUIRD SKILLS: Java SQL Git	Visit Save

IT Job Market Analyzer

Saved vacancies: 2 [Collapse](#)

Number of vacancies shown: 10 [🔊](#) [🔍](#) Experience [▼](#)

[DOU](#) [DJINNI](#) [LINKEDIN](#)

Found 9 vacancies.

01 [DJinni]	Junior PHP Developer (Laravel / Symfony) Not specified partnerdevs Україна Виключно від 0.5 років досвіду	REQUIRD SKILLS: React SQL Docker AWS PHP MongoDB	Visit Delete
02 [LinkedIn]	Data Analyst By agreement. TRX United States Look in the description	REQUIRD SKILLS: SQL	Visit Delete
01 [LinkedIn]	Insights Program Manager by agreement. LinkedIn San Francisco, CA Look in the description	REQUIRD SKILLS: Python SQL	Visit Save



Job analysis

Average salary

Average experience

Top skills

Top locations

Average salary

~2430 USD

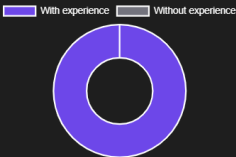
for 5 vacancies from 15



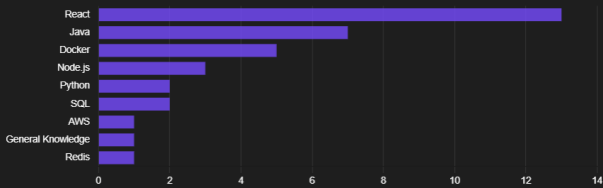
Average experience

~3.0 years

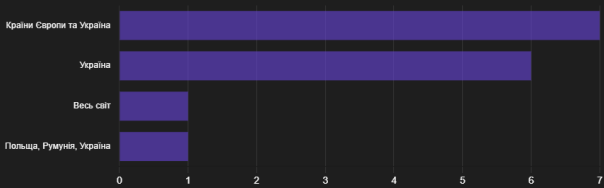
for 15 vacancies from 15



Top skills



Top locations



ВИСНОВКИ

Під час виробничої практики я реалізувала повноцінну бекенд-частину навчального проєкту, який вирішує практичну задачу аналізу ринку IT-вакансій. Під час роботи мені вдалося застосувати такі знання як: патерни Strategy та Facade, принцип Low coupling/High cohesion, multithreading, документація, робота з зовнішніми веб-ресурсами та побудова REST-подібних API. Попри застосування тих знань, які я вже попередньо мала, навчилася працювати і з новими для себе інструментами.

Робота над модулем скрапінгу дозволила глибше зрозуміти особливості взаємодії з різними сайтами та API, тонкощі HTML-парсингу, обмеження швидкодії та важливість обробки помилок. Реалізація фасаду й паралельного збору даних навчила будувати гнучкі та розширювані рішення, які не залежать від конкретних реалізацій скраперів і дозволяють легко підключати нові джерела.

Спільне проєктування дало досвід роботи в команді. Було цікаво постійно обговорювати ідеї, етапи на яких ми знаходились, знаходити разом рішення до тієї чи іншої проблеми.

Загалом, участь у цьому проєкті дозволила мені поглибити знання в області веб-розробки та обробки даних, на практиці застосувати інструменти, які ми вивчали на лекціях, та вивчити нові; отримати досвід роботи в команді.

Я вважаю набуті навички та досвід будуть корисними як для подальшого навчання, так і для старту кар'єри у напрямі бекенд або full-stack розробки.